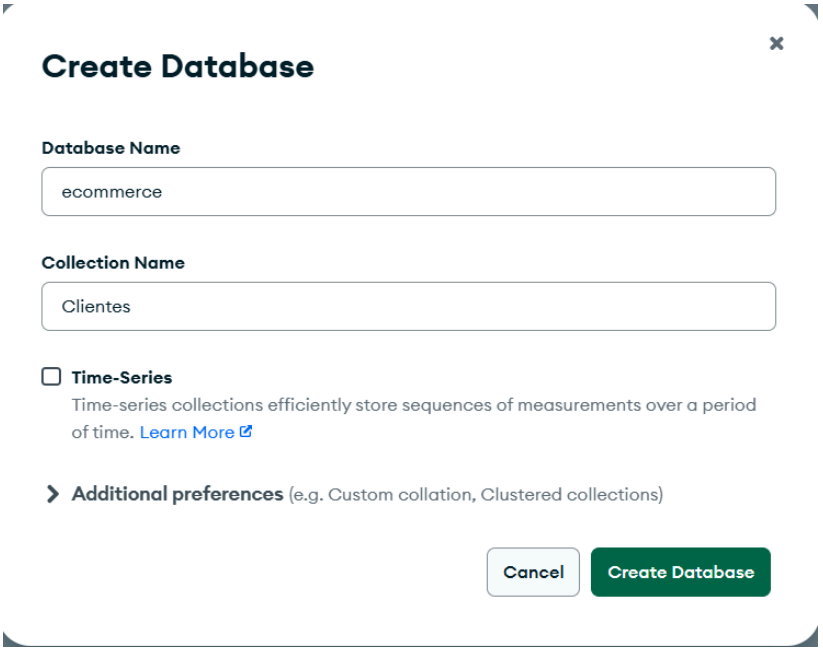
Trabalho Prático - Banco de Dados NoSQL com MongoDB
Equipe: Davi Miguel Cunha e Arthur de França Schoening

1-1) O modelo relacional tradicional organiza os dados em tabelas com linhas e colunas definidas, enquanto o MongoDB utiliza o modelo orientado a documentos flexíveis em formato BSON/JSON organizado em coleções. Um problema enfrentado por desenvolvedores é a chamada “Incompatibilidade de Impedância”, que se caracteriza como a dificuldade de traduzir dados entre o modelo de programação orientada a objetos e o modelo relacional de banco de dados, isto se resolve com o uso de ferramentas extras (como ORMs) para fazer essa conversão. Dessa forma, a utilização de banco de dados orientado a agregados se mostra muito interessante em aplicações modernas, pois alinha o modelo de banco de dados diretamente à estrutura de objetos da aplicação, eliminando a necessidade de junções complexas e mapeamentos.

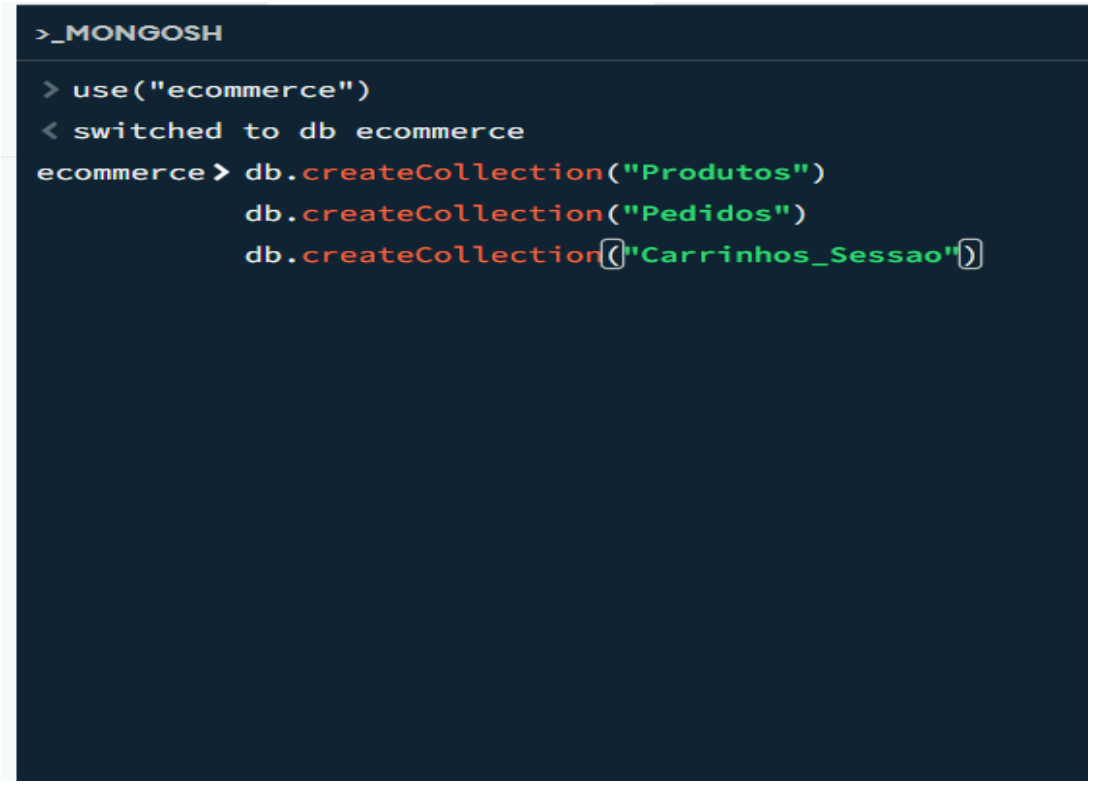
1-2) A persistência poliglota se caracteriza como a utilização de diferentes tecnologias de banco de dados em um único sistema para atender às necessidades específicas de cada tipo de dado, cada tipo de banco é otimizado para atender diferentes objetivos e nenhum atende perfeitamente todas as demandas de um sistema complexo. Por conta disso, é altamente recomendado a implementação dessas diferentes abordagens em um mesmo sistema, o MongoDB possui extrema vantagem em uma funcionalidade de cadastro de produtos, pois possui ampla flexibilidade para cadastrar com atributos completamente diferentes. Em uma funcionalidade de fechamentos de pedidos, banco de dados relacionais (como MySQL e PostgreSQL) oferecem a melhor abordagem, por conta de sua consistência e transações seguras. Em outros casos, como o de um carrinho de compras, é interessante o uso da abordagem chave-valor (como o Redis e DynamoDB), que possui uma alta capacidade de leitura e escrita com expiração automática. O uso dessas abordagens juntas permite construir um sistema altamente escalável, mas sem abrir mão da segurança.

2) Preparação do Ambiente e Operações CRUD

Criação da database e coleção clientes



Criando as outras coleções necessárias



Como ficou

Collection name	Properties	Storage size	Data size	Documents	Avg. document size	Indexes	Total index size
Carrinhos_Sessao	-	4.10 kB	0 B	0	0 B	1	4.10 kB
Clientes	-	4.10 kB	0 B	0	0 B	1	4.10 kB
Pedidos	-	4.10 kB	0 B	0	0 B	1	4.10 kB
Produtos	-	4.10 kB	0 B	0	0 B	1	4.10 kB

2-1)Inserção de Dados (insertOne)

```
>_MONGOSH

> use("ecommerce")
< switched to db ecommerce
> db.Clientes.insertOne({
    nome: "Carlos Eduardo Silva",
    cpf: "888.568.498-01",
    data_nascimento: ISODate("1990-05-20T00:00:00Z"),
    email: "carlos@email.com",
    tipo_cliente: "B2B",
    empresa: "Tecnologia",
    genero: "Masculino"
  })
< {
    acknowledged: true,
    insertedId: ObjectId('6aced5c0ed219c19682284be')
  }
ecommerce >
```

2-2)Atualização com Incremental (updateOne, $set e $inc)

```
> db.Clientes.updateOne(
    { cpf: "888.568.498-01" },
    {
      $set: { tipo_cliente: "B2B" },
      $inc: { total_compras: 500 }
    }
  )
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
ecommerce>
```

2-3)Manipulação de Arrays ($addToSet)

```
> db.Clientes.updateOne(
    { cpf: "888.568.498-01" },
    {
      $addToSet: { categorias_interesse: "Eletrônicos" }
    }
  )
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
ecommerce>
```

2-4)Remoção de Registros (deleteMany)

```
> db.Pedidos.deleteMany({
      valor_total: {$lte: 0}})
< {
    acknowledged: true,
    deletedCount: 0
  }
ecommerce>
```

3-1)Projeção e Ordenação

```
> db.Clientes.find(
    {tipo_cliente:"B2B"},
    {nome:1,email:1,_id:0}).sort
    ({nome:1})
< {
    nome: 'Carlos Eduardo Silva',
    email: 'carlos@email.com'
  }
  {
    nome: 'Carlos Silva',
    email: 'carlos@email.com'
  }
  {
    nome: 'Pedro Santos',
    email: 'pedro@email.com'
  }
ecommerce>
```

3-2)Paginação (limit e skip)

```
> db.Pedidos.find().skip(20).limit(5)
<
ecommerce> Não achou nada pois não tem 20 pedidos
```

3-3)Operadores de Comparação e Datas ($gt)

```
> db.Clientes.find(
      {data_nascimento:{$gt:ISODate("1992-01-01")

      }})
< {
    _id: ObjectId('6aced581f5e7dccbbe85b1b4'),
    nome: 'Ana Souza',
    cpf: '111.222.333-44',
    email: 'ana@email.com',
    tipo_cliente: 'B2C',
    data_nascimento: ISODate('1998-03-15T00:00:00.000Z')
  }
ecommerce>
```

3-4)Operadores Lógicos e de Conjunto ($or e $in)

```
> db.Clientes.find(
     {tipo_cliente:{$in:["B2B","B2C"]

     }})
< {
    _id: ObjectId('6aced581f5e7dccbbe85b1b3'),
    nome: 'Carlos Silva',
    cpf: '888.568.498-01',
    email: 'carlos@email.com',
    tipo_cliente: 'B2B',
    empresa: 'Silva Tecnologia',
    data_nascimento: ISODate('1990-05-20T00:00:00.000Z'),
    total_compras: 500,
    categorias_interesse: [
      'Eletrônicos'
    ]
  }
  {
    _id: ObjectId('6aced581f5e7dccbbe85b1b4'),
    nome: 'Ana Souza',
    cpf: '111.222.333-44',
    email: 'ana@email.com',
    tipo_cliente: 'B2C',
    data_nascimento: ISODate('1998-03-15T00:00:00.000Z')
  }
  {
    _id: ObjectId('6aced581f5e7dccbbe85b1b5'),
    nome: 'Pedro Santos',
```

3-5)Operadores de Elemento ($exists)

```
> db.Clientes.find({empresa:{$exists:true}})
< {
    _id: ObjectId('6aced581f5e7dccbbe85b1b3'),
    nome: 'Carlos Silva',
    cpf: '888.568.498-01',
    email: 'carlos@email.com',
    tipo_cliente: 'B2B',
    empresa: 'Silva Tecnologia',
    data_nascimento: ISODate('1990-05-20T00:00:00.000Z'),
    total_compras: 500,
    categorias_interesse: [
      'Eletrônicos'
    ]
  }
  {
    _id: ObjectId('6aced581f5e7dccbbe85b1b5'),
    nome: 'Pedro Santos',
    cpf: '222.333.444-55',
    email: 'pedro@email.com',
    tipo_cliente: 'B2B',
    empresa: 'Santos Comércio',
    data_nascimento: ISODate('1985-07-10T00:00:00.000Z')
  }
  {
    _id: ObjectId('6aced5c0ed219c19682284be'),
```

3-6) Busca por Padrão de Texto ($regex)

```
> db.Produtos.find(
      {
          categoria:{$regex:"^Eletrônicos"

          }})
< {
    _id: ObjectId('6aced582f5e7dccbbe85b1b6'),
    nome: 'Notebook',
    categoria: 'Eletrônicos',
    preco: 3500,
    especificacoes: {
      memoria: '16GB',
      armazenamento: '512GB SSD'
    }
  }
  {
    _id: ObjectId('6aced582f5e7dccbbe85b1b8'),
    nome: 'Celular',
    categoria: 'Eletrônicos',
    preco: 2000,
    especificacoes: {
      memoria: '8GB',
      armazenamento: '256GB'
    }
  }
ecommerce>
```

4-1)Agrupamento e Métricas Financeiras ($group)

```
> db.Pedidos.aggregate([
    {
      $group: {
        _id: "$tipo_pedido",
        total: { $sum: 1 },
        media: { $avg: "$valor_total" },
        maior: { $max: "$valor_total" }
      }}])
< {
    _id: 'B2B',
    total: 2,
    media: 4500,
    maior: 5000
  }
  {
    _id: 'B2C',
    total: 1,
    media: 100,
    maior: 100
  }
ecommerce >
```

4-2)Desconstrução de Arrays e Contagem ($unwind e $sortByCount)

```
> db.Pedidos.aggregate([
    {$unwind: "$itens"},
    {$sortByCount: "$itens.categoria"}
  ])
< {
    _id: 'Eletrônicos',
    count: 3
  }
  {
    _id: 'Vestuário',
    count: 2
  }
ecommerce>
```

4-3)Filtragem e Agregação Combinada ($match e $group)

```
> db.Pedidos.aggregate([
  {
    $match: {
      status: "Entregue",
      valor_total: { $gt: 3500 }
    }},{
    $group: {
      _id: null,
      total: { $sum: 1 }
    }}])
< {
  _id: null,
  total: 2
}
ecommerce>
```

4-4)Junção entre Coleções ($lookup e $project)

```
> db.Clientes.aggregate([
    {
      $lookup:{
        from: "Pedidos",
        localField: "cpf",
        foreignField: "cpf_cliente",
        as: "clientes_pedidos"
      }},{
      $project:{
        _id: 0,
        data_nascimento: 0,
        genero: 0,
        profissao: 0
      }},
    {$limit: 5}
  ])
< {
    nome: 'Carlos Silva',
    cpf: '888.568.498-01',
    email: 'carlos@email.com',
    tipo_cliente: 'B2B',
    empresa: 'Silva Tecnologia',
    total_compras: 500,
    categorias_interesse: [
      'Eletrônicos'
```